

Progetto OMAT

Tesina tecnica per progetto di maturità

Introduzione

Il progetto OMAT è una piattaforma web pensata per gestire due aree operative collegate a un'azienda o a un laboratorio:

- la raccolta e la gestione delle richieste d'ordine da parte delle aziende;
- la raccolta e la valutazione delle richieste PCTO da parte degli studenti.

Il sistema è diviso in due applicazioni principali:

- **omat_progetto_Client**: interfaccia web realizzata con Angular;
- **omat_progetto_Server**: backend realizzato con Node.js, Express, TypeScript e PostgreSQL/Supabase.

L'obiettivo del progetto è simulare un'applicazione reale con utenti diversi, autenticazione, ruoli, pagine protette, comunicazione client-server e persistenza dei dati su database.

Obiettivi Del Progetto

Il progetto nasce con l'idea di digitalizzare alcuni processi che normalmente richiederebbero moduli cartacei, email o comunicazioni manuali.

Gli obiettivi principali sono:

- permettere alle aziende di registrarsi, accedere e inviare richieste d'ordine;
- permettere agli studenti di inviare richieste PCTO;
- fornire agli amministratori una dashboard per visualizzare e gestire ordini e richieste;
- proteggere le pagine in base al ruolo dell'utente;
- salvare i dati in un database relazionale;
- usare un'architettura separata tra frontend e backend.

Struttura Del Repository

```
progetto_OMAT/  
├── docs/images/           # Screenshot e immagini usate nel README  
├── omat_progetto_Client/  
│   ├── src/app/pages/    # Pagine principali  
│   ├── src/app/core/     # Servizi, modelli e autenticazione  
│   ├── src/app/shared/   # Componenti riutilizzabili  
│   └── static/           # Versione/statica o asset storici del  
sito  
├── omat_progetto_Server/  
│   └── server.ts         # Server Node.js + Express  
                           # API, autenticazione, rotte e avvio
```

```
server
├── src/supabaseClient.ts    # Connessione a Supabase
├── keys/                   # Chiavi locali per JWT/HTTPS
├── db/                     # File di supporto
└── supabase/               # Configurazione Supabase locale
```

Tecnologie Utilizzate

Client

Il client è stato realizzato con:

- Angular 20;
- TypeScript;
- Angular Router;
- Reactive Forms;
- Angular Signals;
- HttpClient;
- componenti standalone;
- CSS personalizzato;
- **lucide-angular** per le icone.

Server

Il server utilizza:

- Node.js;
- Express 5;
- TypeScript;
- PostgreSQL tramite **pg**;
- Supabase tramite **@supabase/supabase-js**;
- JWT per l'autenticazione;
- cookie HTTP-only;
- **bcryptjs** per l'hash delle password;
- CORS con credenziali;
- HTTP e HTTPS in ambiente locale.

Architettura Generale

L'applicazione segue una divisione classica tra frontend e backend.

```
Utente
|
v
Angular Client (porta 4200)
|
| chiamate HTTP con cookie/token
v
```

```
Express API (porta 3001)
|
v
PostgreSQL / Supabase
```

Il client si occupa dell'interfaccia, dei form, della navigazione e della visualizzazione dei dati. Il server gestisce autenticazione, autorizzazioni, salvataggio e lettura dei dati dal database.

Immagini Del Progetto

Schema Del Database

Lo schema mostra le tabelle principali del progetto e le relazioni tra studenti, richieste PCTO, aziende e ordini.

 Schema del database OMAT

Area Studenti: Richiesta PCTO

La pagina PCTO permette allo studente di compilare la richiesta indicando dati personali, scuola, classe, indirizzo di studio, periodo e motivazione.

 Pagina richiesta PCTO

Dashboard Amministrativa

La dashboard admin raccoglie le informazioni principali su ordini e PCTO, mostrando statistiche, filtri e collegamenti ai dettagli delle richieste.

 Dashboard amministrativa OMAT

Area Aziende: Richiesta Ordine

La pagina degli ordini permette all'azienda di caricare un disegno tecnico, compilare i dettagli della lavorazione e inviare la richiesta.

 Pagina richiesta ordine

Ruoli Degli Utenti

Il progetto prevede tre ruoli:

Ruolo	Funzione
azienda	Invia richieste d'ordine e visualizza i propri ordini
studente	Invia richieste PCTO
admin	Visualizza e gestisce ordini e richieste PCTO

Questa distinzione è importante perché permette di mostrare pagine diverse in base all'utente connesso.

Funzionalità Principali

Login E Registrazione

Gli utenti possono registrarsi come:

- azienda;
- studente;
- amministratore.

Durante la registrazione la password viene salvata in forma protetta tramite hash. Durante il login il server controlla se l'email appartiene a un admin, a un'azienda o a uno studente, poi genera un token JWT.

Richieste Ordini

Una azienda può compilare un form con:

- nome del prodotto;
- materiale;
- quantità;
- priorità;
- descrizione;
- note aggiuntive;
- eventuali allegati.

L'ordine viene poi salvato nel database e può essere visualizzato dall'admin nella dashboard.

Richieste PCTO

Uno studente può inviare una richiesta PCTO indicando:

- dati personali;
- scuola;
- classe;
- indirizzo di studio;
- periodo richiesto;
- motivazione.

Il server gestisce anche il caso in cui lo studente esista già nel database, evitando duplicazioni inutili.

Dashboard Admin

La dashboard amministrativa permette di:

- visualizzare gli ordini;
- filtrare gli ordini per stato;
- cercare per codice, cliente, materiale o admin assegnato;
- visualizzare le richieste PCTO;
- filtrare le richieste PCTO per stato;
- aprire le pagine di dettaglio;
- aggiornare lo stato delle pratiche.

API Principali

Metodo	Endpoint	Descrizione
GET	/api/health	Controlla lo stato del server e del database
POST	/api/auth/login	Login utente
POST	/api/auth/logout	Logout utente
POST	/api/auth/register-company	Registrazione azienda
POST	/api/auth/register-student	Registrazione studente
POST	/api/auth/register-admin	Registrazione admin
GET	/api/orders	Lista ordini visibili all'utente
POST	/api/orders	Creazione ordine
PATCH	/api/orders/:id/status	Aggiornamento stato ordine, solo admin
GET	/api/pcto	Lista richieste PCTO, solo admin
POST	/api/pcto	Creazione richiesta PCTO
PATCH	/api/pcto/:id/status	Aggiornamento stato PCTO, solo admin

Parti Di Codice Significative

1. Protezione Delle Rotte Angular

Nel client le rotte vengono protette in base al ruolo dell'utente. Per esempio, la pagina degli ordini è accessibile solo alle aziende, mentre la dashboard è accessibile solo agli admin.

```
export const routes: Routes = [  
  { path: 'pcto', component: PctoComponent, canActivate:  
    [requireRole(['studente'])] },  
  { path: 'richieste-ordini', component: RichiesteOrdiniComponent,  
    canActivate: [requireRole(['azienda'])] },  
  { path: 'admin', component: AdminDashboardComponent, canActivate:  
    [requireRole(['admin'])] },  
];
```

Questa parte è interessante perché mostra come il frontend non sia composto solo da pagine, ma anche da regole di navigazione.

2. Guard Per Controllare Il Ruolo

Il guard controlla il ruolo salvato nello stato di autenticazione. Se l'utente non ha i permessi necessari, viene rimandato alla pagina di login.

```
export function requireRole(roles: UserRole[]): CanActivateFn {
  return () => {
    const auth = inject(AuthStateService);
    const router = inject(Router);
    const role = auth.role();

    if (role && roles.includes(role)) {
      return true;
    }

    return router.createUrlTree(['/login']);
  };
}
```

3. Stato Di Autenticazione Con Angular Signals

Il client usa i signal di Angular per conservare l'utente connesso e calcolare automaticamente informazioni derivate, come il ruolo o lo stato di login.

```
private readonly userSignal = signal<AuthUser | null>(
  this.readStoredUser());

readonly user = this.userSignal.asReadonly();
readonly isLoggedIn = computed(() => Boolean(this.userSignal()));
readonly role = computed(() => this.userSignal()?.role ?? null);
```

Questa scelta rende il codice più reattivo: quando cambia l'utente, anche l'interfaccia può aggiornarsi in modo automatico.

4. Servizio API Centralizzato

Il client non chiama il backend direttamente dai componenti, ma usa un servizio dedicato. Questo rende il codice più ordinato e facilita eventuali modifiche future agli endpoint.

```
@Injectable({ providedIn: 'root' })
export class OmatApiService {
  private readonly http = inject(HttpClient);
  private readonly baseUrl = 'http://127.0.0.1:3001/api';

  login(email: string, password: string): Observable<LoginResponse> {
    return this.http.post<LoginResponse>(
      `${this.baseUrl}/auth/login`,
      { email, password },
      { withCredentials: true },
    );
  }

  getOrders(): Observable<OrderRequest[]> {
```

```
    return this.http.get<OrderRequest[]>(`${this.baseUrl}/orders`, {
      withCredentials: true });
  }
}
```

5. Login Con Controllo Su Piu Tabelle

Nel server il login cerca l'utente tra admin, aziende e studenti. Quando trova una corrispondenza valida, genera il token JWT con il ruolo corretto.

```
if (admin && (await verifyPassword(password, admin.password))) {
  const payload: TokenPayload = {
    id: Number(admin.idLavoratore),
    email: admin.email,
    role: 'admin',
    name: `${admin.nome} ${admin.cognome}`.trim(),
  };

  const token = createToken(payload);
  res.cookie('TOKEN', token, cookieOptions);
  res.send({ token, user: payload });
  return;
}
```

Questa parte e' utile da presentare perche collega autenticazione, database, sicurezza e gestione dei ruoli.

6. Middleware Di Autenticazione JWT

Il middleware `requireAuth` controlla la presenza del token, lo verifica e salva i dati dell'utente dentro la request.

```
function requireAuth(req: AuthenticatedRequest, res: Response, next:
NextFunction): void {
  const bearerToken = req.headers.authorization?.startsWith('Bearer ')
    ? req.headers.authorization.slice('Bearer '.length)
    : undefined;
  const token = req.cookies?.TOKEN ?? bearerToken;

  if (!token) {
    res.status(401).send('Token mancante');
    return;
  }

  jwt.verify(token, jwtKey, (err: any, payload: any) => {
    if (err) {
      res.status(401).send('Token non valido o scaduto');
      return;
    }
  })
}
```

```
    req.user = toTokenPayload(payload);
    res.cookie('TOKEN', createToken(req.user), cookieOptions);
    next();
  });
}
```

L'aspetto importante è che il server non si fida del client: ogni richiesta protetta viene controllata prima di accedere ai dati.

7. Controllo Dei Permessi Admin

Alcune operazioni, come aggiornare lo stato di un ordine o visualizzare tutte le richieste PCTO, sono riservate agli amministratori.

```
function requireAdmin(req: AuthenticatedRequest, res: Response, next:
NextFunction): void {
  if (req.user?.role !== 'admin') {
    res.status(403).send('Permessi insufficienti');
    return;
  }

  next();
}
```

Questo middleware viene usato insieme a `requireAuth`, creando una protezione a due livelli: prima autenticazione, poi autorizzazione.

8. Transazione Per La Richiesta PCTO

La creazione di una richiesta PCTO usa una transazione. In questo modo, se qualcosa va storto, il database torna allo stato precedente.

```
const client = await pool.connect();

try {
  await client.query('begin');

  // ricerca o creazione dello studente
  // inserimento della richiesta PCTO

  await client.query('commit');
} catch (err) {
  await client.query('rollback');
  throw err;
} finally {
  client.release();
}
```


Questa e una parte particolarmente importante dal punto di vista tecnico, perche mostra attenzione all'integrita dei dati.

9. Dashboard Admin Con Filtri Reattivi

Nel client la dashboard usa `computed` per filtrare gli ordini in base alla ricerca e allo stato selezionato.

```
protected readonly filteredOrders = computed(() => {
  const search = this.searchTerm().trim().toLowerCase();
  const status = this.statusFilter();

  return this.orders().filter((order) => {
    const matchesStatus = status === 'all' || order.status === status;
    const matchesSearch =
      !search ||
      [order.code, order.title, order.customer, order.material,
order.assignedAdmin].some((value) =>
        value.toLowerCase().includes(search),
      );

    return matchesStatus && matchesSearch;
  });
});
```

Questo codice mostra come la logica dell'interfaccia sia dinamica: cambiando filtro o testo di ricerca, la lista si aggiorna senza ricaricare la pagina.

Database

Il server usa PostgreSQL e crea o aggiorna automaticamente alcune tabelle fondamentali, tra cui:

- `ordini`;
- `richiestePCTO`.

Le altre tabelle principali usate dal login sono:

- `admin`;
- `aziende`;
- `studenti`.

Alcuni campi importanti sono:

- `stato`, per indicare l'avanzamento di ordini e richieste;
- `idAzienda`, per collegare un ordine a una azienda;
- `idStudiante`, per collegare una richiesta PCTO a uno studente;
- `dataAggiornamento`, per registrare l'ultima modifica.

Sicurezza

Il progetto integra diverse scelte di sicurezza:

- password salvate con hash tramite `bcryptjs`;
- token JWT per riconoscere l'utente dopo il login;
- cookie HTTP-only per ridurre l'esposizione del token lato JavaScript;
- rotte backend protette con middleware;
- rotte frontend protette con guard Angular;
- query SQL parametrizzate per ridurre il rischio di SQL injection;
- controllo dei ruoli per separare studenti, aziende e admin.

Come Avviare Il Progetto

1. Avvio Del Server

```
cd omat_progetto_Server  
npm install  
npm start
```

Il server HTTP parte sulla porta `3001`.

Per il corretto funzionamento servono le variabili d'ambiente del database, ad esempio:

```
DATABASE_URL=postgresql://utente:password@host:porta/database  
JWT_SECRET=una_chiave_segreta  
PORT=3001  
HTTPS_PORT=3000
```

Se si usa Supabase, servono anche:

```
SUPABASE_URL=...  
SUPABASE_SERVICE_ROLE_KEY=...
```

2. Avvio Del Client

```
cd omat_progetto_Client  
npm install  
npm start
```

Il client Angular parte normalmente su:

```
http://127.0.0.1:4200
```

Possibili Miglioramenti Futuri

Il progetto puo essere ampliato con:

- upload reale dei file allegati agli ordini;
- notifiche email per cambio stato;
- pannello admin piu dettagliato con statistiche;
- validazioni backend piu complete sui dati ricevuti;
- gestione profilo utente;
- refresh token separato dal token di accesso;
- deployment online del client e del server;
- test automatici per API e componenti Angular.

Conclusione

OMAT e un progetto completo per una tesina di maturita perché unisce aspetti di frontend, backend, database e sicurezza. Non si limita a mostrare pagine statiche, ma implementa un flusso reale: registrazione, login, ruoli, invio richieste, dashboard amministrativa e aggiornamento degli stati.

Dal punto di vista didattico permette di spiegare concetti importanti come architettura client-server, autenticazione JWT, hash delle password, protezione delle rotte, chiamate HTTP, database relazionale e gestione dello stato nel frontend.